

UNIVERSIDAD EL BOSQUE

Ingeniería de Sistemas
Bases de Datos I

DOCUMENTO DE IMPLEMENTACIÓN DE RDBMS

Angela Maria Reina Vivas
Angel Felipe Alvarez Benavides
Cristian David Vargas Cetina

Asignatura: Bases de Datos I
Docente: Christian Felipe Duarte Arevalo

Bogotá D.C.
2026

1. INTRODUCCIÓN

El presente documento describe la implementación del Sistema Gestor de Bases de Datos Relacional (RDBMS) seleccionado para el proyecto GymDB: PostgreSQL 18. En él se presentan aspectos como la instalación y configuración del motor de base de datos, el diseño del esquema relacional, la integración con el backend desarrollado en Spring Boot mediante Spring Data JPA e Hibernate, el mapeo objeto-relacional de las entidades Java y la configuración de la conexión a la base de datos.

Asimismo, el documento funciona como referencia técnica del proceso de implementación de la capa de persistencia del sistema, facilitando futuras tareas de mantenimiento, extensión o replicación del entorno de desarrollo.

2. AMBIENTE DE IMPLEMENTACIÓN

2.1 Especificaciones del Entorno de Desarrollo

Componente	Versión / Especificación
Motor de Base de Datos	PostgreSQL 18.x
Puerto de escucha	5432 (puerto estándar PostgreSQL)
Sistema Operativo	Windows 11
Lenguaje Backend	Java 17 (LTS)
Framework Backend	Spring Boot 3.3.x
ORM / JPA Provider	Spring Data JPA + Hibernate 6.x
Driver JDBC	org.postgresql:postgresql:42.x
IDE de desarrollo	IntelliJ IDEA Community Edition 2024.x
Herramienta de BD	pgAdmin 4

3. INSTALACIÓN DE POSTGRESQL

3.1 Descarga e Instalación en Windows

PostgreSQL para Windows se descargó desde el sitio oficial de EnterpriseDB, distribuidor oficial de PostgreSQL para sistemas Windows:

[EnterpriseDB Downloads](#)

El proceso de instalación consistió en ejecutar el instalador de PostgreSQL 18 para Windows x64 y seleccionar los componentes principales del sistema, incluyendo PostgreSQL Server y pgAdmin 4. Durante la instalación se configuró el puerto estándar 5432 y se definió el usuario administrador postgres para el entorno de desarrollo.

Finalmente, PostgreSQL quedó registrado como servicio del sistema operativo, permitiendo su ejecución automática al iniciar Windows.

3.2 Verificación de la Instalación

Para verificar el correcto funcionamiento de PostgreSQL, se ejecutó el siguiente comando desde la línea de comandos de Windows (CMD o PowerShell):

- `psql -U postgres -h localhost -p 54`

Una vez establecida la conexión con el servidor, se utilizó la siguiente consulta para verificar la versión instalada del motor de base de datos:

- `SELECT version();`

Resultado obtenido:

- PostgreSQL 18.3 on x86_64-windows, 64-bit

La ejecución exitosa de esta consulta confirmó que el servidor PostgreSQL se encontraba correctamente instalado y operativo en el entorno de desarrollo.

4. CREACIÓN DE LA BASE DE DATOS

4.1 Creación de la Base de Datos GYM

Una vez instalado PostgreSQL, se creó la base de datos del proyecto con el nombre gym, siguiendo las convenciones de nomenclatura en minúsculas utilizadas comúnmente en PostgreSQL. La creación pudo realizarse tanto desde pgAdmin 4 como desde el cliente de línea de comandos psql. Una vez instalado PostgreSQL, se creó la base de datos del proyecto con el nombre gym, siguiendo las convenciones de nomenclatura en minúsculas utilizadas comúnmente en PostgreSQL. La creación pudo realizarse tanto desde pgAdmin 4 como desde el cliente de línea de comandos psql.

Mediante psql:

```
-- Conectarse al servidor PostgreSQL
psql -U postgres
```

```
-- Crear la base de datos del proyecto
CREATE DATABASE gym;
```

```
-- Conectarse a la base de datos creada
\c gym
```

Mediante pgAdmin 4

1. En el panel lateral, expandir el servidor PostgreSQL.

2. Hacer clic derecho sobre **Databases** → **Create** → **Database**.
3. Ingresar el nombre de la base de datos: gym.
4. Guardar la configuración para completar la creación.

5.DISEÑO E IMPLEMENTACIÓN DEL ESQUEMA RELACIONAL

5.1 Diagrama Entidad–Relación Conceptual

El modelo de datos del sistema Gym está compuesto por 12 tablas organizadas en tres áreas funcionales:

- Gestión de personas: cliente, instructor
- Gestión de clases y asistencia: clase, sesion, asistencia
- Gestión financiera: plan, suscripcion, factura, pago, efectivo, tarjeta, transferencia

Las relaciones entre entidades fueron definidas de acuerdo con los requerimientos funcionales del sistema:

- cliente (1) $\longleftrightarrow$ (N) suscripcion: un cliente puede registrar múltiples suscripciones a lo largo del tiempo.
- plan (1) $\longleftrightarrow$ (N) suscripcion: un plan puede estar asociado a múltiples suscripciones.
- suscripcion (1) $\longleftrightarrow$ (N) factura: una suscripción puede generar múltiples facturas.
- factura (1) $\longleftrightarrow$ (1) pago: cada factura registra un único pago asociado.
- pago $\longleftrightarrow$ efectivo / tarjeta / transferencia: el sistema permite registrar distintos métodos de pago.
- clase (1) $\longleftrightarrow$ (N) sesion: una clase puede tener múltiples sesiones programadas.
- instructor (1) $\longleftrightarrow$ (N) sesion: un instructor puede dirigir múltiples sesiones.
- sesion (1) $\longleftrightarrow$ (N) asistencia: una sesión registra múltiples asistencias.
- cliente (1) $\longleftrightarrow$ (N) asistencia: un cliente puede registrar múltiples asistencias.

5.2 Script de Creación del Esquema (DDL)

El esquema de la base de datos fue creado mediante el siguiente script DDL ejecutado en la base de datos gym. Las tablas fueron creadas siguiendo el orden de dependencias definido por las claves foráneas del modelo relacional.

5.2.1 Tabla: cliente

La tabla cliente almacena la información personal de los usuarios registrados en el gimnasio. El campo numero_documento posee una restricción de unicidad (UNIQUE) para evitar registros duplicados.

```
CREATE TABLE cliente (  
  id_cliente SERIAL PRIMARY KEY,  
  primer_nombre VARCHAR(50) NOT NULL,  
  segundo_nombre VARCHAR(50),  
  primer_apellido VARCHAR(50) NOT NULL,  
  segundo_apellido VARCHAR(50),  
  numero_documento VARCHAR(20) UNIQUE NOT NULL,  
  tipo_documento VARCHAR(10) NOT NULL,  
  telefono VARCHAR(20),  
  email VARCHAR(100)  
);
```

5.2.2 Tabla: instructor

La tabla instructor registra la información personal y profesional de los instructores del gimnasio. Su estructura es similar a la tabla cliente, incorporando adicionalmente el campo especialidad.

```
CREATE TABLE instructor (  
  id_instructor SERIAL PRIMARY KEY,  
  primer_nombre VARCHAR(50) NOT NULL,  
  segundo_nombre VARCHAR(50),  
  primer_apellido VARCHAR(50) NOT NULL,  
  segundo_apellido VARCHAR(50),  
  numero_documento VARCHAR(20) UNIQUE NOT NULL,  
  tipo_documento VARCHAR(10) NOT NULL,  
  telefono VARCHAR(20),  
  email VARCHAR(100),  
  especialidad VARCHAR(100)  
);
```

5.2.3 Tabla: plan

La tabla plan define los planes de membresía ofrecidos por el gimnasio, incluyendo su costo y duración.

```
CREATE TABLE plan (  
  id_plan SERIAL PRIMARY KEY,  
  nombre_plan VARCHAR(100) NOT NULL,  
  costo DECIMAL(10,2) NOT NULL,  
  duracion_dias INTEGER  
);
```

5.2.4 Tabla: suscripcion

La tabla suscripcion registra las suscripciones de los clientes a los diferentes planes del gimnasio. El campo costo permite conservar el valor registrado al momento de la suscripción, incluso si el precio del plan cambia posteriormente.

```
CREATE TABLE suscripcion (  
  id_suscripcion SERIAL PRIMARY KEY,  
  fecha_inicio DATE NOT NULL,  
  fecha_fin DATE NOT NULL,
```

```

    costo      DECIMAL(10,2),
    beneficios  TEXT,
    estado      VARCHAR(20),
    id_cliente  INT REFERENCES cliente(id_cliente),
    id_plan     INT REFERENCES plan(id_plan)
);

```

5.2.5 Tabla: factura

La tabla factura almacena las facturas generadas a partir de las suscripciones registradas en el sistema. El campo estado permite identificar el estado actual de la factura (PENDIENTE, PAGADA, VENCIDA o ANULADA).

```

CREATE TABLE factura (
    id_factura  SERIAL PRIMARY KEY,
    fecha_emision  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    total_factura DECIMAL(10,2) NOT NULL,
    estado      VARCHAR(20),
    id_suscripcion INT REFERENCES suscripcion(id_suscripcion)
);

```

5.2.6 Tabla: pago

La tabla pago registra los pagos realizados por los clientes asociados a las facturas generadas. La restricción UNIQUE sobre el campo id_factura garantiza que cada factura tenga asociado un único pago.

```

CREATE TABLE pago (
    id_pago     SERIAL PRIMARY KEY,
    total_recibo DECIMAL(10,2) NOT NULL,
    fecha       TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    id_factura  INT UNIQUE REFERENCES factura(id_factura)
);

```

5.2.7 Tablas de Métodos de Pago: efectivo, tarjeta, transferencia

Las tablas efectivo, tarjeta y transferencia permiten registrar los distintos métodos de pago soportados por el sistema.

```

CREATE TABLE efectivo (
    id_efectivo SERIAL PRIMARY KEY,
    monto       DECIMAL(10,2) NOT NULL,
    id_pago     INT REFERENCES pago(id_pago)
);

```

```

CREATE TABLE tarjeta (
    id_tarjeta  SERIAL PRIMARY KEY,
    monto       DECIMAL(10,2) NOT NULL,
    franquicia  VARCHAR(30),
    tipo        VARCHAR(20),
    id_pago     INT REFERENCES pago(id_pago)
);

```

```

CREATE TABLE transferencia (
    id_transferencia SERIAL PRIMARY KEY,

```

```

    monto          DECIMAL(10,2) NOT NULL,
    banco_origen    VARCHAR(50),
    numero_transaccion VARCHAR(50),
    id_pago          INT REFERENCES pago(id_pago)
);

```

5.2.8 Tablas de Clases y Sesiones

Las siguientes tablas gestionan la programación de clases, sesiones y el registro de asistencia de los clientes.

```

CREATE TABLE clase (
    id_clase    SERIAL PRIMARY KEY,
    nombre_clase VARCHAR(100) NOT NULL,
    descripcion TEXT
);

CREATE TABLE sesion (
    id_sesion    SERIAL PRIMARY KEY,
    fecha        DATE NOT NULL,
    hora_inicio  TIME,
    hora_fin     TIME,
    cupo_max     INT NOT NULL,
    cupos_disponibles INT NOT NULL,
    id_clase     INT REFERENCES clase(id_clase),
    id_instructor INT REFERENCES instructor(id_instructor)
);

CREATE TABLE asistencia (
    id_asistencia SERIAL PRIMARY KEY,
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    estado_asistencia VARCHAR(20),
    id_cliente     INT REFERENCES cliente(id_cliente),
    id_sesion      INT REFERENCES sesion(id_sesion)
);

```

6. DESCRIPCIÓN DETALLADA DE LAS TABLAS

Tabla	PK	Tipo PK	FK(s)	Descripción
cliente	id_cliente	SERIAL	—	Datos personales de clientes. UNIQUE en numero_documento.
instructor	id_instructor	SERIAL	—	Datos personales e info profesional (especialidad). UNIQUE en numero_documento.
plan	id_plan	SERIAL	—	Planes de membresía con nombre, costo y duración en días.

suscripcion	id_suscripcion	SERIAL	cliente, plan	Historial de suscripciones con fechas, estado y costo histórico.
factura	id_factura	SERIAL	suscripcion	Facturas emitidas. Estado: PENDIENTE, PAGADA, VENCIDA, ANULADA.
pago	id_pago	SERIAL	factura (UNIQUE)	Registro del pago asociado a una factura.
efectivo	id_efectivo	SERIAL	pago	Registros de pagos realizados en efectivo.
tarjeta	id_tarjeta	SERIAL	pago	Almacena información de franquicia y tipo de tarjeta.
transferencia	id_transferencia	SERIAL	pago	Pagos por transferencia bancaria. Almacena banco origen y número de transacción.
clase	id_clase	SERIAL	—	Catálogo de tipos de clase (Yoga, Spinning, etc.) con descripción.
sesion	id_sesion	SERIAL	clase, instructor	Sesiones programadas con fecha, horario, cupo máximo y disponible.
asistencia	id_asistencia	SERIAL	cliente, sesion	Registro de asistencia de clientes a sesiones programadas.

7. INTEGRACIÓN CON SPRING BOOT Y SPRING DATA JPA

7.1 Dependencias de Maven

El proyecto fue desarrollado utilizando Spring Boot 3.3.2 y Java 17. Las siguientes dependencias Maven fueron utilizadas para la construcción de servicios REST, la integración con PostgreSQL y la persistencia de datos mediante Spring Data JPA.

```
<dependencies>
```

```
    <!-- Servicios REST -->
```

```
    <dependency>
```

```
        <groupId>org.springframework.boot</groupId>
```

```
        <artifactId>spring-boot-starter-web</artifactId>
```

```
    </dependency>
```

```
    <!-- Persistencia con Spring Data JPA -->
```

```
    <dependency>
```

```
        <groupId>org.springframework.boot</groupId>
```

```
        <artifactId>spring-boot-starter-data-jpa</artifactId>
```

```
    </dependency>
```



```

<!-- Driver JDBC PostgreSQL -->
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
</dependency>

<!-- Validación de datos -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<!-- Dependencias para pruebas -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>

</dependencies>

```

7.2 Configuración de application.properties

El archivo application.properties contiene la configuración utilizada para establecer la conexión con PostgreSQL y definir el comportamiento de JPA/Hibernate dentro del proyecto Spring Boot.

```

# Identificación de la aplicación
spring.application.name=Gym

# Configuración de conexión PostgreSQL
spring.datasource.url=jdbc:postgresql://localhost:5432/gym
spring.datasource.username=postgres
spring.datasource.password=${DB_PASSWORD}
spring.datasource.driver-class-name=org.postgresql.Driver

# Configuración JPA / Hibernate
spring.jpa.hibernate.ddl-auto=update
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

```

Análisis de cada parámetro de configuración:

Parámetro	Descripción y Justificación
-----------	-----------------------------

spring.datasource.url	Define la URL de conexión JDBC hacia la base de datos PostgreSQL local gym.
spring.datasource.username	Usuario utilizado para la conexión con PostgreSQL.
spring.datasource.password	Contraseña de acceso a la base de datos. En entornos reales se recomienda utilizar variables de entorno para proteger credenciales sensibles.
spring.datasource.driver-class-name	Driver JDBC utilizado para la conexión con PostgreSQL.
spring.jpa.hibernate.ddl-auto=update	Permite que Hibernate actualice automáticamente el esquema de la base de datos según las entidades definidas en el proyecto.
spring.jpa.database-platform	Muestra en consola las consultas SQL generadas por Hibernate durante la ejecución de la aplicación.
spring.jpa.show-sql=true	Registra los valores de los parámetros de binding en las sentencias SQL preparadas, permitiendo ver exactamente qué valores se están enviando a la base de datos.
spring.jpa.properties.hibernate.format_sql=true	Formatea las consultas SQL impresas en consola para facilitar su lectura.

8. MAPEO OBJETO-RELACIONAL (ORM)

8.1 Descripción General del Mapeo

El mapeo entre las tablas de PostgreSQL y las clases Java del proyecto se realiza mediante anotaciones JPA (Jakarta Persistence API). Spring Data JPA, junto con Hibernate como implementación ORM, permite convertir las operaciones realizadas sobre objetos Java en consultas SQL compatibles con PostgreSQL.

A continuación, se describe el mapeo objeto-relacional utilizado para las principales entidades del sistema.

8.2 Entidad Cliente ↔ Tabla cliente

```
@Entity
@Table(name = "cliente")
public class Cliente {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_cliente")
    private Long idCliente;

    @Column(name = "numero_documento", unique = true, nullable = false)
    private String numeroDocumento;
```

```

@Column(name = "primer_nombre", nullable = false)
private String primerNombre;

// ... otros campos ...

@JsonIgnore
@OneToMany(mappedBy = "cliente", cascade = CascadeType.ALL)
private List<Asistencia> asistencias;

@JsonIgnore
@OneToMany(mappedBy = "cliente")
private List<Suscripcion> suscripciones;
}

```

8.3 Entidad Asistencia ↔ Tabla asistencia

La entidad Asistencia ilustra el uso de `@Enumerated(EnumType.STRING)` para almacenar enumeraciones Java como valores de tipo texto en PostgreSQL.

```

@Entity
@Table(name = "asistencia")
public class Asistencia {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long idAsistencia;

    @Column(name = "fecha_registro")
    private LocalDateTime fechaRegistro = LocalDateTime.now();

    @Enumerated(EnumType.STRING)
    @Column(name = "estado_asistencia")
    private EstadoAsistencia estadoAsistencia = EstadoAsistencia.PRESENTE;

    @JsonIgnoreProperties({"suscripciones", "asistencias"})
    @ManyToOne
    @JoinColumn(name = "id_cliente")
    private Cliente cliente;

    @JsonIgnoreProperties({"asistencias"})
    @ManyToOne
    @JoinColumn(name = "id_sesion")
    private Sesion sesion;
}

```

8.4 Entidad Factura ↔ Tabla factura

```

@Entity
@Table(name = "factura")
public class Factura {

```

```

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = "id_factura")
private Long idFactura;

@Column(name = "fecha_emision")
private LocalDateTime fechaEmision = LocalDateTime.now();

@Column(name = "total_factura")
private Double totalFactura;

@Enumerated(EnumType.STRING)
private EstadoFactura estado = EstadoFactura.PENDIENTE;

@JsonIgnoreProperties({"facturas", "cliente", "plan"})
@ManyToOne
@JoinColumn(name = "id_suscripcion")
private Suscripcion suscripcion;
}

```

8.5 Entidad Pago ↔ Tabla pago (relación OneToOne con Factura)

La entidad Pago implementa una relación uno a uno (@OneToOne) con la entidad Factura.

```

@Entity
@Table(name = "pago")
public class Pago {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_pago")
    private Long idPago;

    @Column(name = "total_recibo")
    private Double totalRecibo;

    private LocalDateTime fecha = LocalDateTime.now();

    @OneToOne
    @JoinColumn(name = "id_factura")
    private Factura factura;
}

```

8.6 Entidades de Método de Pago: Tarjeta

La entidad Tarjeta utiliza enumeraciones (enum) para representar la franquicia y el tipo de tarjeta asociados al pago.

```

@Entity
@Table(name = "tarjeta")
public class Tarjeta {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long idTarjeta;

    private Double monto;

    @Enumerated(EnumType.STRING)
    private FranquiciaTarjeta franquicia;

    @Enumerated(EnumType.STRING)
    private TipoTarjeta tipo;

    @ManyToOne
    @JoinColumn(name = "id_pago")
    private Pago pago;
}

```

8.7 Tabla de Mapeo Completo de Tipos

Tipo Java	Tipo PostgreSQL	Anotación JPA	Observación
Long	INTEGER / SERIAL	@Id @GeneratedValue(IDENTITY)	Identificadores autogenerados
String	VARCHAR(n) / TEXT	@Column	Utilizado para nombres, descripciones y campos de texto
Double	DOUBLE	@Column	Utilizado para valores monetarios y montos
Integer	INTEGER	@Column	Utilizado para cupos y duración de planes
LocalDate	DATE	@Column	Fechas sin componente de hora
LocalTime	TIME	@Column	Horarios de sesiones
LocalDateTime	TIMESTAMP	@Column	Registro de fecha y hora en facturas,

			pagos y asistencias
Enum Java	VARCHAR(n)	@Enumerated(EnumType.STRING)	Almacenamiento textual de estados y tipos
List<Entidad>	FK en tabla hija	@OneToMany(mappedBy=...)	Relación uno a muchos
Entidad	Columna FK INT REFERENCES	@ManyToOne @JoinColumn	Relación muchos a uno

9. CAPA DE REPOSITORIOS Y CONSULTAS NATIVAS

9.1 Arquitectura de Repositorios

Los repositorios del sistema extienden la interfaz `JpaRepository<Entidad, Long>`, lo que permite utilizar operaciones CRUD básicas como guardar, consultar, actualizar y eliminar registros sin implementar manualmente dichas operaciones.

Para consultas específicas del sistema se utilizaron distintos mecanismos proporcionados por Spring Data JPA:

- Métodos derivados del nombre, por ejemplo: `findByNumeroDocumento()` o `findByPago()`.
- Consultas mediante `@Query` utilizando JPQL (Java Persistence Query Language).
- Consultas SQL nativas mediante `@Query(nativeQuery = true)` para operaciones específicas sobre PostgreSQL.

9.2 Consultas Nativas Destacadas

9.2.1 Ranking de Clientes por Asistencia

```
-- ClienteRepository.rankingClientesAsistencia()
```

```
SELECT a.id_cliente, COUNT('X')
FROM asistencia a
GROUP BY a.id_cliente
ORDER BY COUNT(*) DESC;
```

9.2.2 Distribución de Métodos de Pago

```
-- PagoRepository.distribucionMetodosPago()
```

```
SELECT 'EFECTIVO' AS metodo, COUNT('X')
FROM efectivo

UNION ALL

SELECT 'TARJETA', COUNT(*)
FROM tarjeta
```

```
UNION ALL
```

```
SELECT 'TRANSFERENCIA', COUNT('X')  
FROM transferencia;
```

9.2.3 Ranking de Clientes por Total Pagado

```
-- PagoRepository rankingClientesPagos()  
  
SELECT c.id_cliente,  
       c.primer_nombre,  
       SUM(p.total_recibo) AS total_pagado  
FROM pago p  
JOIN factura f ON p.id_factura = f.id_factura  
JOIN suscripcion s ON f.id_suscripcion = s.id_suscripcion  
JOIN cliente c ON s.id_cliente = c.id_cliente  
GROUP BY c.id_cliente, c.primer_nombre  
ORDER BY total_pagado DESC;
```

9.2.4 Búsqueda Case-Insensitive de Instructores (ILIKE)

```
-- InstructorRepository.buscarPorNombre()  
  
SELECT *  
FROM instructor  
WHERE primer_nombre ILIKE CONCAT('%', :nombre, '%');
```

10. IMPLEMENTACIÓN DEL SERVICIO TRANSACCIONAL

10.1 Pago Mixto con @Transactional

El proceso de registro de pagos utiliza la anotación @Transactional de Spring para asegurar que todas las operaciones relacionadas con un pago se ejecuten de manera consistente dentro de una misma transacción.

```
@Service  
public class PagoService {  
  
    @Transactional  
    public void registrarPagoMixto(PagoDTO pagoDTO) {  
  
        // Verificar existencia de factura  
        Factura factura = facturaRepository.findById(pagoDTO.idFactura())  
            .orElseThrow(() -> new RuntimeException("Factura no encontrada"));  
  
        // Verificar que la factura no haya sido pagada  
        if (pagoRepository.findByFactura(factura).isPresent()) {  
            throw new RuntimeException("La factura ya fue pagada");  
        }  
  
        // Validar total del pago  
        double suma = pagoDTO.metodos().stream()  
            .mapToDouble(DetalleMetodoDTO::monto)  
            .sum();  
    }  
}
```

```

        if (Math.abs(suma - pagoDTO.totalRecibo()) > 0.01) {
            throw new RuntimeException("El total no coincide");
        }

        // Registrar pago principal
        Pago nuevoPago = new Pago();
        nuevoPago.setTotalRecibo(pagoDTO.totalRecibo());
        nuevoPago.setFactura(factura);

        pagoRepository.save(nuevoPago);

        // Registrar métodos de pago
        for (DetalleMetodoDTO detalle : pagoDTO.metodos()) {

            switch (detalle.tipoMetodo().toUpperCase()) {

                case "EFECTIVO" ->
                    efectivoRepository.save(new Efectivo(...));

                case "TARJETA" ->
                    tarjetaRepository.save(tarjeta);

                case "TRANSFERENCIA" ->
                    transferenciaRepository.save(tr);
            }
        }

        // Actualizar estado de factura
        factura.setEstado(EstadoFactura.PAGADA);
        facturaRepository.save(factura);
    }
}

```

La anotación `@Transactional` garantiza que, si ocurre un error durante cualquiera de las operaciones del proceso, los cambios realizados no se almacenarán parcialmente en la base de datos.

11. DATOS DE PRUEBA (SEED DATA)

Para facilitar las pruebas del sistema, se propone el siguiente conjunto de datos de prueba que puede ejecutarse en la base de datos gym:

-- — Insertar Planes

```

INSERT INTO plan (nombre_plan, costo, duracion_dias) VALUES
('Plan Básico', 79000.00, 30),
('Plan Estándar', 129000.00, 30),
('Plan Premium', 199000.00, 30),
('Plan Trimestral', 349000.00, 90),
('Plan Anual', 999000.00, 365);

```



```

-- — Insertar Instructores
INSERT INTO instructor
(primer_nombre, primer_apellido, numero_documento,
tipo_documento, especialidad, email) VALUES
('Carlos', 'Mendoza', '80123456', 'CC', 'Yoga y Meditación',
'carlos.mendoza@gym.co'),
('Laura', 'Torres', '52987654', 'CC', 'Spinning y Cardio',
'laura.torres@gym.co'),
('Jorge', 'Ruiz', '79345678', 'CC', 'Crossfit y Funcional',
'jorge.ruiz@gym.co'),
('Diana', 'Castro', '43876543', 'CC', 'Pilates y Stretching',
'diana.castro@gym.co');

-- — Insertar Clases
INSERT INTO clase (nombre_clase, descripcion) VALUES
('Yoga Básico', 'Clase introductoria de yoga para todos los niveles'),
('Spinning Intenso', 'Ciclismo estacionario de alta intensidad'),
('CrossFit', 'Entrenamiento funcional de alta intensidad'),
('Pilates Core', 'Fortalecimiento del núcleo y flexibilidad'),
('Zumba', 'Baile aeróbico al ritmo de música latina');

-- — Insertar Clientes
INSERT INTO cliente
(primer_nombre, primer_apellido, numero_documento,
tipo_documento, telefono, email) VALUES
('Ana María', 'Gómez', '1030123456', 'CC', '3101234567',
'ana.gomez@email.com'),
('Pedro', 'Vargas', '1020987654', 'CC', '3209876543',
'pedro.vargas@email.com'),
('Sofía', 'Rojas', '1040345678', 'CC', '3156789012',
'sofia.rojas@email.com');

```

12. CONSULTAS DE VERIFICACIÓN Y MONITOREO

Las siguientes consultas permiten validar la estructura de la base de datos y verificar información general del sistema.

12.1 Verificación de Tablas

```

-- Listar tablas del esquema público
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name;

-- Verificar claves foráneas
SELECT

```

```

tc.table_name AS tabla,
kcu.column_name AS columna,
ccu.table_name AS tabla_referenciada
FROM information_schema.table_constraints AS tc
JOIN information_schema.key_column_usage AS kcu
  ON tc.constraint_name = kcu.constraint_name
JOIN information_schema.constraint_column_usage AS ccu
  ON ccu.constraint_name = tc.constraint_name
WHERE tc.constraint_type = 'FOREIGN KEY';

```

12.2 Consultas de Negocio para Verificación

```

-- Totales registrados por entidad
SELECT
  (SELECT COUNT(*) FROM cliente) AS total_clientes,
  (SELECT COUNT(*) FROM instructor) AS total_instructores,
  (SELECT COUNT(*) FROM plan) AS total_planes,
  (SELECT COUNT(*) FROM suscripcion) AS total_suscripciones,
  (SELECT COUNT(*) FROM factura) AS total_facturas,
  (SELECT COUNT(*) FROM pago) AS total_pagos,
  (SELECT COUNT(*) FROM sesion) AS total_sesiones,
  (SELECT COUNT(*) FROM asistencia) AS total_asistencias;

-- Total recaudado por método de pago
SELECT 'Efectivo' AS metodo, COALESCE(SUM(monto), 0)
FROM efectivo

UNION ALL

SELECT 'Tarjeta', COALESCE(SUM(monto), 0)
FROM tarjeta

UNION ALL

SELECT 'Transferencia', COALESCE(SUM(monto), 0)
FROM transferencia;

-- Facturas pendientes de pago
SELECT
  f.id_factura,
  f.total_factura,
  f.fecha_emision,
  c.primer_nombre || ' ' || c.primer_apellido AS cliente
FROM factura f
JOIN suscripcion s
  ON f.id_suscripcion = s.id_suscripcion
JOIN cliente c
  ON s.id_cliente = c.id_cliente
WHERE f.estado = 'PENDIENTE'
ORDER BY f.fecha_emision ASC;

```

13. CONSIDERACIONES DE SEGURIDAD Y BUENAS PRÁCTICAS

13.1 Gestión de Credenciales

En el entorno de desarrollo del proyecto, las credenciales de conexión a PostgreSQL se configuraron directamente en el archivo `application.properties`, utilizando el usuario local `postgres`.

Para un entorno de producción, se recomienda utilizar variables de entorno o herramientas de gestión segura de credenciales para evitar exponer información sensible dentro del código fuente.

- `spring.datasource.password=${DB_PASSWORD}`

13.2 Configuración ddl-auto para Producción

La propiedad `spring.jpa.hibernate.ddl-auto=update` es adecuada para desarrollo activo, pero en producción se recomienda:

- `ddl-auto=validate`: Hibernate valida que el esquema existente sea compatible con las entidades Java, pero no realiza cambios. Falla al inicio si hay incompatibilidades.
- `ddl-auto=none`: Hibernate no toca el esquema. Los cambios se gestionan con herramientas de migración como Flyway o Liquibase.
- Para producción con Flyway: `spring.flyway.enabled=true` con scripts de migración versionados (`V1__init.sql`, `V2__add_column.sql`, etc.).

13.3 Pool de Conexiones

Spring Boot utiliza HikariCP como pool de conexiones por defecto para la gestión eficiente de conexiones JDBC hacia PostgreSQL. Para este proyecto académico se utilizó la configuración predeterminada proporcionada por Spring Boot.

14. CONCLUSIÓN

La implementación del RDBMS PostgreSQL 16 en el proyecto GymDB ha sido completada de manera exitosa, abarcando la instalación del motor, la creación de la base de datos gym, la definición completa del esquema relacional con 12 tablas interrelacionadas, y la integración con el backend Spring Boot 3.x mediante Spring Data JPA e Hibernate 6.x.

La configuración implementada demuestra un uso del stack tecnológico: el dialecto `PostgreSQLDialect` de Hibernate genera SQL optimizado para PostgreSQL, el uso de `@Transactional` garantiza la atomicidad de las operaciones financieras críticas (pagos mixtos), y las consultas nativas (`@Query nativeQuery=true`) aprovechan características exclusivas de PostgreSQL como el operador `ILIKE` y el casting explícito para comparaciones de enums almacenados como `VARCHAR`.

El esquema relacional diseñado cumple con las reglas del negocio establecidas en los requisitos del sistema: la restricción UNIQUE en pago.id_factura garantiza la relación 1:1 entre pagos y facturas, las claves foráneas establecen la integridad referencial entre todas las entidades, y el modelo de tablas separadas para métodos de pago (efectivo, tarjeta, transferencia) permite el soporte de pagos mixtos con múltiples métodos en una sola transacción.

Este documento constituye la referencia técnica completa de la capa de persistencia del sistema, y puede ser utilizado como base para la replicación del ambiente de desarrollo, la extensión del sistema con nuevas entidades, o la migración a un entorno de producción en la nube.